# 11-Builder Pattern

The **Builder Pattern** is a *creational design pattern* used in object-oriented programming to construct complex objects step by step. It separates the construction process from the final representation, so the same building process can create different representations.

ពេលយើងបង្កើត Object ម្តង់ៗពី Constructor វាត្រូវការអោយយើងធ្វើការ កំនត់ទាំងអស់នៅក្នុង Object នឹង។ ឧ បង្កើត object student ត្រូវមាន Name, age, gender, address etc. តែក្នុងលក្ខខណ្ឌ ណាមួយដែលយើងចង់អោយវាធ្វើការ បង្កើតតែ file តែមួយចំនួនយើងអាចធ្វើការ build ជា Builder Patter បាន

## 🔧 Why use the Builder Pattern?

Use it when:

- An object has **many optional parameters**
- Construction is **complex or requires multiple steps**
- You want **immutable objects**
- You want to avoid constructors with too many parameters (a.k.a. *telescoping constructor problem*)

---

## 🧠 Core Idea

Instead of creating an object in one big constructor, you:

1. Create a **Builder**
2. Set properties step by step
3. Call `build()` to get the final object

---

## 📐 Structure

Typical components:

- **Product** → the object being built
- **Builder** → interface or abstract class
- **Concrete Builder** → actual implementation
- **Director (optional)** → controls construction order

## Example (Java)

Java

```java
class User {
    private String name;
    private int age;
    private String email;

    private User(Builder builder) {
        this.name = builder.name;
        this.age = builder.age;
        this.email = builder.email;
    }

    public static class Builder {
        private String name;
        private int age;
        private String email;

        public Builder setName(String name) {
            this.name = name;
            return this;
        }

        public Builder setAge(int age) {
            this.age = age;
            return this;
        }

        public Builder setEmail(String email) {
            this.email = email;
            return this;
        }

        public User build() {
            return new User(this);
        }
    }
}
```

- Class User have field: name, age and email. ដើម្បីបង្កើត Builder យើងធ្វើការ return វាមួយៗ នឹងចុងក្រោយ ត្រូវធ្វើការ called build.

**Usage:**

```java
User user = new User.Builder()
                    .setName("Alice")
                    .setAge(25)
                    .setEmail("alice@example.com")
                    .build();
```

---

## ✅ Advantages

- Improves **readability**
- Avoids complex constructors
- Supports **immutable objects**
- Flexible object creation

---

## ❌ Disadvantages

- Requires more code (extra classes)
- Overkill for simple objects

---

## 📌 Real-world examples

- `StringBuilder` in Java
- Configuration objects (e.g., HTTP clients)
- UI builders (like dialog builders)

- ចុងក្រោយយើងគ្រាន់តែ ហៅ Method .build ដើម្បីបង្កើត Object នោះតែប៉ុណ្ណោះ